

CS201 · MID-SEM

THE PAPER

No past mid-sem paper for this one exists in the folder — so this page does not pretend to read a pattern off thin air. Instead: an honest map of what the lecture slides actually weight, and a **full predicted paper** built from that, worked out line by line.

WHAT ACTUALLY GETS ASKED

READ OFF THE SLIDES, NOT OFF OLD PAPERS

Every other subject on this site got its prediction by reading three or four old papers. Here there are none, so the method has to change. What we have instead is the complete set of lecture slides — seven decks plus the new inheritance deck — and slide decks leak their own priorities: the topics that get worked examples get asked, and the topics that get one bullet point do not.

TOPIC	DECK	MARKS
Constructors, destructors and the order they run in	MAT - 5	4–5
Classes, objects, member functions, access specifiers	MAT - 4	4–5
Inheritance — modes, ambiguity, virtual base	NEW DECK	6–8
Function overloading and default arguments	MAT - 3	3
Call by value, address and reference	MAT - 3	3
Friend functions	MAT - 4	3
Operator overloading	MAT - 6	4
The copy constructor, shallow versus deep	MAT - 5	3–4
static data members and member functions	MAT - 4	3
new / delete and the scope resolution operator	MAT - 2	3

TOPIC	DECK	MARKS
POP versus OOP, the six core ideas	MAT - 1	2-3
Tokens, data types, reference variables	MAT - 2	2
Inline functions, recursion	MAT - 3	2
String manipulation using overloaded operators	MAT - 6	0-3

TINTED ROWS ARE THE ONES WE WOULD NOT SIT THE EXAM WITHOUT · MARKS ARE OUR ESTIMATE OUT OF 30

READ IT THIS WAY

Programming papers are not theory papers. Look at what the decks spend their pages on: almost every concept arrives attached to a *complete runnable program*, and the slides walk through its output. That is a very strong signal. The four question types that fall out of it — and that we have built the predicted paper from — are:

1. **“Predict the output, with justification.”** The single most common shape. It is cheap to set, impossible to bluff, and it tests three things at once: the language rule, the order of execution, and whether you actually read the code. Constructor/destructor ordering and static members are the classic vehicle.
2. **“Find and correct the errors.”** Usually four or five planted mistakes in a twenty-line program — a missing `ClassName::`, a private member touched from `main()`, a call to a constructor that does not exist. One mark per error found *and* fixed; naming the line without the fix gets half.
3. **“Write a class that...”** The long one. Expect to declare a class, give it a constructor and a destructor, overload something, and demonstrate it in `main()`.
4. **“Distinguish between X and Y.”** Member function vs friend function, call by value vs reference, overloading vs overriding, structure vs class. Answer in a table — it is faster to write and faster to mark.

Where the marks have moved. The inheritance deck is new material and it is dense: three inheritance modes, the visibility table, single / multilevel / multiple / hierarchical / hybrid, the ambiguity problem, and the virtual base class. That is a whole question’s worth of content and it has never been examined here before. Our prediction gives it 6 of 30, and if anything that is conservative.

PREDICTED PAPER

OUR READING, WITH FULL SOLUTIONS

This is a prediction, and there is no past paper behind it. It is built from the lecture decks, from the shape the other four subjects' mid-sems take at this institute, and from what the slides chose to spend their worked examples on. Treat it as a very good revision set rather than as a forecast. Full solutions throughout, and every program has been traced by hand line by line.

S.V. National Institute of Technology, Surat-7

B.Tech. – Object Oriented Programming

MID-SEM EXAMINATION — *predicted*

CS-201 OBJECT ORIENTED PROGRAMMING

MARKS: 30

TIME: 1.5 HRS

ASSUME NECESSARY DATA WHEREVER REQUIRED

Instruction. (i) Assume necessary header files are included wherever required. (ii) The figure to the right indicates marks. (iii) Justify every predicted output — an unexplained answer carries half credit.

Q-1 Predict the exact output of the following program and justify the order in which
(a) the lines appear.

04

SOLUTION

```

#include <iostream>
using namespace std;

class Item {
    static int count;
    int id;
public:
    Item() { id = ++count; cout << "ctor " << id << endl; }
    ~Item() { cout << "dctor " << id << endl; --count; }
    static int live() { return count; }
};

int Item::count = 0;           // definition outside the class

int main() {
    Item a;
    {
        Item b, c;
        cout << "inside : " << Item::live() << endl;
    }
    cout << "outside: " << Item::live() << endl;
    return 0;
}

```

Trace it line by line.

STATEMENT	COUNT	PRINTED
<code>int Item::count = 0;</code>	0	—
<code>Item a;</code> → <code>id = ++count = 1</code>	1	<code>ctor 1</code>
<code>Item b;</code> → <code>id = 2</code>	2	<code>ctor 2</code>
<code>Item c;</code> → <code>id = 3</code>	3	<code>ctor 3</code>
<code>Item::live()</code>	3	<code>inside : 3</code>
block ends → c destroyed first	2	<code>dctor 3</code>
then b destroyed	1	<code>dctor 2</code>
<code>Item::live()</code>	1	<code>outside: 1</code>
<code>main()</code> returns → a destroyed	0	<code>dctor 1</code>

```

ctor 1
ctor 2
ctor 3
inside : 3
dctor 3
dctor 2
outside: 1
dctor 1

```

Eight lines, in exactly that order

The three justifications the question is asking for.

1. **One count for the whole class.** `count` is `static`, so it is not one variable per object — it is a single variable shared by every `Item` ever made. That is why the ids come out 1, 2, 3 instead of every object thinking it is number 1.
2. **Destruction is the reverse of construction.** Objects on the stack are destroyed in **last in, first out** order, so inside the block it is c before b. This is why you see `dctor 3` before `dctor 2`, never the other way round.
3. **The inner block has its own scope.** b and c die at the closing brace, not at the end of `main()` — which is the whole point of the printed counts, 3 inside and 1 outside.

THE LINE PEOPLE FORGET

`int Item::count = 0;` outside the class is **not optional**. Declaring a static member inside the class only says it exists; it must be **defined** once at file scope or the linker fails with “undefined reference to `Item::count`”. Note also that the definition does *not* repeat the word `static`, and it is allowed to touch a private member because it is part of the class.

And `live()` is `static`, so it is called as `Item::live()` with no object at all — a static member function has no `this` pointer and therefore cannot touch `id`.

Q-1
(b)

The following program does not compile. Identify **every** error, give the line, say why it is an error, and write the corrected line.

03

SOLUTION

```

1  #include <iostream>
2  using namespace std;
3
4  class Box {
5      int len;
6  public:
7      Box(int l) { len = l; }
8      int area();
9  };
10
11 int area() { return len * len; }
12
13 int main() {
14     Box b;
15     b.len = 5;
16     cout << b.area;
17     return 0;
18 }

```

LINE	WHAT IS WRONG	CORRECTION
11	The function is defined without the class name, so it is a <i>global</i> function, not <code>Box</code> 's member — and a global function has no access to <code>len</code> . Two errors for the price of one.	<code>int Box::area() { return len * len; }</code>
14	<code>Box b;</code> calls a default constructor , but the moment you write <i>any</i> constructor the compiler stops supplying one.	<code>Box b(5);</code> — or add <code>Box() { len = 0; }</code> to the class
15	<code>len</code> is private . Nothing outside the class may touch it, and <code>main()</code> is outside the class.	Delete the line — <code>b</code> was built with 5 already. Or add a public <code>void set(int l){ len = l; }</code> and call <code>b.set(5);</code>
16	<code>b.area</code> is the function itself, not a call. A member function name without <code>()</code> cannot be streamed and cannot decay to a pointer.	<code>cout << b.area() << endl;</code>

The corrected program:

```

class Box {
    int len;
public:
    Box(int l) { len = l; }
    int area();
};

int Box::area() { return len * len; }

int main() {
    Box b(5);
    cout << b.area() << endl;    // prints 25
    return 0;
}

```

Four errors – lines 11, 14, 15, 16. Corrected output: 25

Q-2 (a) *Attempt any three of (a) to (d). Predict the output. Explain the three parameter-passing mechanisms it demonstrates and state when each one should be used.* 03

SOLUTION

```

void f1(int x) { x = x + 10; }
void f2(int *x) { *x = *x + 10; }
void f3(int &x) { x = x + 10; }

int main() {
    int a = 5, b = 5, c = 5;
    f1(a);
    f2(&b);
    f3(c);
    cout << a << " " << b << " " << c << endl;
    return 0;
}

```

5 15 15

Output: 5 15 15

	CALL BY VALUE F1(INT X)	CALL BY ADDRESS F2(INT *X)	CALL BY REFERENCE F3(INT &X)
WHAT IS PASSED	A copy of the value	The address , itself by value	An alias for the original

	CALL BY VALUE F1(INT X)	CALL BY ADDRESS F2(INT *X)	CALL BY REFERENCE F3(INT &X)
CALLER'S VARIABLE	Unchanged — a stays 5	Changed — b becomes 15	Changed — c becomes 15
CALL LOOKS LIKE	f1(a)	f2(&b) — the & gives it away	f3(c) — looks identical to call by value
INSIDE, YOU WRITE	x	*x, every single time	x
CAN BE NULL?	—	Yes — must be checked	No — a reference always refers to something

When to use which.

- **Value** — for small built-in types you do not intend to modify. It is the safest, because the function physically cannot damage the caller's data.
- **Reference** — when the function must modify the argument, and for large objects you want to pass without copying. Add `const ( f(const Big &x) )` when you want the speed of a reference with the safety of a value.
- **Address** — when the argument is genuinely optional (so a null pointer is meaningful), or when interfacing with C.

The exam point: references are a **C++ addition** — C has only value and address. That is why `f3`'s call site looks exactly like `f1`'s: the caller cannot tell, from the call alone, that `c` is about to be modified. Convenient, and occasionally dangerous.

Q-2
(b)

(i) The two declarations below cause a compilation error at one of the calls. Which call, and why? (ii) For the overload set that follows, state which function each call selects and give the rule.

03

SOLUTION

(i) Overloading against default arguments.

```
void show(int a, int b = 10);    // can be called with ONE argument
void show(int a);               // can also be called with ONE argument

show(5, 6);                    // fine
show(5);                       // ERROR
```

show(5) is ambiguous

Why. A default argument does not create a second function — it lets the *existing* function be called with fewer arguments. So `show(5)` is a perfect, exact match for **both** declarations: for the first with `b` defaulted to 10, and for the second directly. Neither is better than the other, so the compiler cannot choose and reports the call as ambiguous.

Note where the error appears: **at the call, not at the declaration**. Declaring the two overloads is legal — they have different signatures. The program only breaks the moment you write a one-argument call.

The fix is to remove the clash, either by deleting `void show(int)` , or by dropping the default so the first one always needs two arguments.

(ii) Overload resolution.

```
void f(int);
void f(double);

f(10);      →
f(3.5);     →
f('A');     →
f(3.5f);    →
```

CALL	SELECTS	RULE
f(10)	f(int)	Exact match — always wins, nothing else is even considered.
f(3. 5)	f(doubl e)	Exact match — an unsuffixed literal like 3.5 is a <code>double</code> , not a <code>float</code> .
f('A)	f(int)	Promotion beats conversion. <code>char</code> → <code>int</code> is an integral promotion; <code>char</code> → <code>double</code> is a standard conversion. Promotion ranks higher, so <code>f(int)</code> wins and prints whatever 65 means to it.
f(3. 5f)	f(doubl e)	Same rule, other direction — <code>float</code> → <code>double</code> is a promotion, <code>float</code> → <code>int</code> is a conversion (and a lossy one).

THE ORDER THE COMPILER WORKS IN

Exact match → **promotion** → **standard conversion** → **user-defined conversion** → **ellipsis**. It stops at the first level where exactly one candidate matches. If a level produces two candidates and no tie-break, the call is **ambiguous** — that is the error in part (i), and it is by far the most common overloading mistake.

And the rule that catches people out: **the return type is not part of the signature**. `int f(int);` and `double f(int);` are not overloads, they are a redeclaration conflict.

Q-2
(c)

Write a program in which a single function accesses the private data of **two different** classes. Explain why the forward declaration is necessary and list the properties of a friend function.

03

SOLUTION

```
#include <iostream>
using namespace std;

class B;                                // forward declaration -- required

class A {
    int x;
public:
    A(int v) { x = v; }
    friend int sum(A, B);                // B is only a NAME here, and that
    // is enough
};

class B {
    int y;
public:
    B(int v) { y = v; }
    friend int sum(A, B);
};

int sum(A a, B b) {                     // note: no 'friend', no 'A::'
    return a.x + b.y;                   // reaches inside both classes
}

int main() {
    A a(10);
    B b(20);
    cout << sum(a, b) << endl;
    return 0;
}
```

30

Output: 30

Why the forward declaration is necessary. Class `A` is compiled first, and its friend declaration `friend int sum(A, B);` mentions `B` — a name the compiler has not met yet. `class B;` on the line above is an **incomplete type declaration**: it promises “a class called `B` exists, its definition is coming”. That is all the compiler needs in order to parse the parameter list. Without it, the friend declaration fails with “`B` has not been declared”.

An incomplete type is enough to *name* in a declaration, but not enough to use: you could not write `B b;` or `b.y` before `B`’s full definition. In this program `sum`’s *body* appears after both classes are complete, so by then everything is known.

Properties of a friend function — the list the examiner wants.

- It is **not a member** of the class, even though it is declared inside it. It is not in the class’s scope.
- It is **called like an ordinary function**, without an object — `sum(a, b)`, never `a.sum(b)`.
- It has no **this** pointer, so every object it works on must be passed as an argument. That is why `sum` takes two parameters where a member function would take one.
- It can be declared in the **public or the private** section — it makes no difference at all to the meaning.
- It can access both **private and protected** members of the class that granted the friendship.
- **Friendship is granted, not taken.** A function cannot declare itself a friend; the class must invite it. This is what keeps encapsulation intact — you can always find every outsider with access by reading the class.
- **Friendship is not inherited and not mutual.** If `A` befriends `B`, `B` does not automatically befriend `A`, and `A`’s derived classes do not inherit its friends.

WHY FRIENDS EXIST AT ALL

It looks like a hole in encapsulation, and it is — a deliberate one. Two situations need it: a function that must reach inside **two unrelated classes**, as here; and an operator whose **left operand is not an object of your class** — which is exactly why `operator<<` is always a friend, since its left operand is a `cout`.

Q-2 Predict the output. Explain the role of the scope resolution operator in this program, and state the consequence of writing `delete arr;` in place of `delete[] arr;` . 03

SOLUTION

```
#include <iostream>
using namespace std;

int m = 100;                                // global

int main() {
    int m = 10;                             // local -- hides the global
    cout << m << " " << ::m << endl;

    int *p = new int(7);
    int *arr = new int[3];
    arr[0] = 1; arr[1] = 2; arr[2] = 3;

    cout << *p << " " << arr[1] << endl;

    delete p;
    delete[] arr;
    return 0;
}
```

```
10 100
7 2
```

Two lines: "10 100" then "7 2"

The scope resolution operator, both of its jobs.

1. **Unary `::m` — reach the global.** The local `m` hides the global one throughout `main()`, so plain `m` is 10. Writing `::m` with nothing to its left says “the `m` at file scope”, which is 100. In C this is impossible — a hidden global is simply unreachable — and it is one of the small additions C++ makes.
2. **Binary `Class::member` — name a member outside its class.** The same operator is what lets you define a member function outside the class body (`int Box::area() {...}`), define a static data member (`int Item::count = 0;`), call a static member function (`Item::live()`), and disambiguate an inherited name (`p.M::display()`).

`delete arr;` instead of `delete[] arr;`. It is **undefined behaviour**. `new[]` and `delete[]` are a matched pair: `new[]` records how many elements it allocated, and `delete[]` reads that count so it can run the destructor on *every* element before releasing the block. Plain `delete` assumes a single object, so it:

- destroys **only the first element** — the other objects' destructors never run, leaking whatever they owned;
- hands the wrong block back to the allocator, which typically **corrupts the heap**.

Here the elements are plain `int` s with no destructor, so on most compilers it happens to survive — which makes it the worst kind of bug: correct-looking, silent, and fatal the moment the array holds real objects. **Whatever you allocate with `new[]`, release with `delete[]`.**

Q-3
(a)

Write a class `Complex` that overloads the `+` operator as a member function, and show its use. Then explain why `c = a + 2;` can be made to work while `c = 2 + a;` cannot, and how a friend function fixes it.

04

SOLUTION

```

#include <iostream>
using namespace std;

class Complex {
    float re, im;
public:
    Complex() { re = im = 0; }
    Complex(float r, float i) { re = r; im = i; }
    Complex(float r) { re = r; im = 0; } // one-arg: also
    a CONVERTER

    Complex operator+(Complex c) { // 'this' is the LEFT operand
        Complex t;
        t.re = re + c.re;
        t.im = im + c.im;
        return t;
    }

    void show() { cout << re << " + j" << im << endl; }
};

int main() {
    Complex a(2.5, 3.5), b(1.5, 2.5), c;
    c = a + b;
    c.show();
    return 0;
}

```

4 + j6

Output: 4 + j6

How the call is actually made. `a + b` is rewritten by the compiler as

```
a.operator+(b)
```

so the **left operand becomes the object** and the right one becomes the argument. A binary operator overloaded as a member therefore takes only **one** parameter — the other operand is `*this`. As a friend it would take **two**.

Why `a + 2` works. There is a one-argument constructor `Complex(float)`, so when the compiler sees `a.operator+(2)` and the parameter wants a `Complex`, it is allowed to build a temporary `Complex(2)` and pass that. A one-argument constructor is a **converting constructor**, whether you meant it to be or not.

Why `2 + a` does not. The same rewrite gives

```
2 . operator+(a)      ← nonsense
```

The left operand is an `int`, and an `int` is a built-in type with no member functions. **The compiler will never convert the left operand of a member operator**, because it has to find the function *before* it can consider conversions, and it looks for it inside the type of the left operand. So the call fails.

The friend fix. Declare the operator as a non-member, and now *both* operands are ordinary parameters, so both are open to conversion:

```
class Complex {
    ...
    friend Complex operator+(Complex, Complex);
};

Complex operator+(Complex p, Complex q) {
    return Complex(p.re + q.re, p.im + q.im);
}

// now all three of these compile:
c = a + b;
c = a + 2;      // 2 → Complex(2)
c = 2 + a;      // 2 → Complex(2) ← this is what the member version
                  could not do
```

MEMBER OR FRIEND? THE ONE-LINE RULE

If the left operand may be something other than your class, it must be a friend.

That is why `operator<<` and `operator>>` are always friends — their left operand is a stream, not your object.

The exceptions run the other way: `=`, `[]`, `()` and `->` **must** be members, by language rule. And the operators you cannot overload at all are `.`, `.*`, `::`, `:::` and `sizeof`.

Q-3
(b)

Predict the output of the program below and state the **two** defects it has.
Rewrite the class so that both are removed.

04

SOLUTION

```

#include <iostream>
#include <cstring>
using namespace std;

class Str {
    char *p;
public:
    Str(const char *s) {
        p = new char[strlen(s) + 1];
        strcpy(p, s);
    }
    ~Str() { delete[] p; }
    void set(int i, char c) { p[i] = c; }
    void show() { cout << p << endl; }
};

int main() {
    Str a("HELLO");
    Str b = a;           // no copy constructor written
    b.set(0, 'J');
    a.show();
    b.show();
    return 0;
}

```

The output.

```

JELLO
JELLO

```

...then the program crashes or corrupts the heap on exit.

Both objects print JELLO – changing b changed a

Why. No copy constructor was written, so the compiler supplies one, and the compiler-supplied version does a **member-by-member copy**. `Str` has exactly one member, the pointer `p`, so what gets copied is the **address**:

```

a.p ----+
      |
      +----> [ H E L L O \0 ]    ONE buffer
      |
b.p ----+

```

That is a **shallow copy**. `b.set(0, 'J')` writes through `b.p`, but `a.p` points at the same bytes, so `a` changes too.

Defect 1 — aliasing. Two objects that were supposed to be independent share one buffer, so modifying either one silently modifies the other.

Defect 2 — the double delete. At the end of `main()`, `b` is destroyed and runs `delete[] p`. Then `a` is destroyed and runs `delete[]` on **the same address**. Releasing the same block twice is **undefined behaviour** — in practice a heap corruption or a crash. This is the more serious of the two, because it can strike long after the real mistake.

The fix — a deep copy constructor, and the assignment operator to match:

```
class Str {
    char *p;
public:
    Str(const char *s) {
        p = new char[strlen(s) + 1];
        strcpy(p, s);
    }

    // DEEP copy constructor -- its own buffer
    Str(const Str &s) {
        p = new char[strlen(s.p) + 1];
        strcpy(p, s.p);
    }

    // and the assignment operator, or 'a = b;' has the same bug
    Str& operator=(const Str &s) {
        if (this != &s) { // guard against a = a;
            delete[] p;
            p = new char[strlen(s.p) + 1];
            strcpy(p, s.p);
        }
        return *this; // so that a = b = c; works
    }

    ~Str() { delete[] p; }
    void set(int i, char c) { p[i] = c; }
    void show() { cout << p << endl; }
};
```

```
with the fix:    HELLO
                  JELLO          and no crash
```

WHY THE PARAMETER MUST BE A REFERENCE

The copy constructor is declared `Str(const Str &s)` and the `&` is **compulsory**. If you wrote `Str(Str s)`, passing the argument by value would itself require a copy — which would call the copy constructor — which would need to copy its argument — forever. The compiler rejects it outright.

The rule to quote: the moment a class owns a resource through a raw pointer, it needs **all three** of a destructor, a copy constructor and a copy assignment operator. Write one and you almost certainly need the other two.

Q-4
(a)

Give the visibility table for the three modes of inheritance. For the code below, state the accessibility of `a`, `b` and `c` inside `B`, inside `C`, and from `main()`.

03

SOLUTION

```
class A {
private:   int a;
protected: int b;
public:   int c;
};

class B : protected A { };      // protected inheritance
class C : private B { };       // private inheritance
```

The visibility table. Read it as: a base member of *this* kind, inherited in *that* mode, becomes...

MEMBER IN THE BASE	PUBLIC INHERITANCE	PROTECTED INHERITANCE	PRIVATE INHERITANCE
PRIVATE	Not inherited — never accessible in any derived class, in any mode		
PROTECTED	protected	protected	private
PUBLIC	public	protected	private

The one-line way to remember it: the inheritance mode is a **ceiling**, not a floor. A member can never come out *more* accessible than the mode allows; if it was already less accessible, it stays where it is. And a private base member is never inherited at all — not even privately.

Applying it to the code.

	A (PRIVATE IN A)	B (PROTECTED IN A)	C (PUBLIC IN A)
INSIDE B B : PROTECTED A	Inaccessible	protected	protected — demoted from public
INSIDE C C : PRIVATE B	Inaccessible	private — demoted again	private
FROM MAIN()	No	No	No — it was public in A, but two modes have shut it away

Nothing from A is reachable through a C object in main()

Two things worth adding.

- A private member of A is *physically present* in every B and C object — it occupies space — but no derived class may name it. The only route to it is a public or protected member function of A.
- Members of C *can* still use `b` and `c`, because private members are accessible inside their own class. What C cannot do is pass them on: a class derived from C gets nothing.

THE DEFAULT NOBODY REMEMBERS

If you leave the mode out, it depends on the keyword you used: `class D : B` is **private** inheritance, but `struct D : B` is **public** inheritance. Write the mode explicitly, every time. Nearly all real inheritance is `public`, because that is the only mode that models “is-a”.

Q-4
(b)

Explain, with code, the ambiguity that arises in multiple inheritance and the two distinct situations in which it occurs. Give the fix for each.

03

SOLUTION

Situation 1 — two bases with the same member name.

```

class M { public: void display() { cout << "M" << endl; } };
class N { public: void display() { cout << "N" << endl; } };

class P : public M, public N { };

int main() {
    P p;
    p.display();          // ERROR -- which display()?
}

```

Both bases contribute a `display()`, neither is preferred, so the compiler reports **“request for member `display` is ambiguous”**. Note that the *class* `P` compiles perfectly — the error appears only at the call.

Fix — the scope resolution operator:

```

p.M::display();          // prints M
p.N::display();          // prints N

```

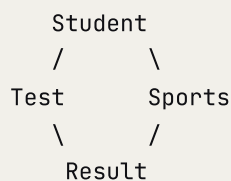
Or, better, resolve it once inside the class so that users of `P` never see the problem:

```

class P : public M, public N {
public:
    void display() { M::display(); }    // P makes the choice
};

```

Situation 2 — the diamond, where the ambiguity is in the *data*.



```

class Student { protected: int roll; };

class Test    : public Student { protected: int m1, m2; };
class Sports  : public Student { protected: int score; };

class Result : public Test, public Sports {
public:
    void show() { cout << roll; }    // ERROR -- which roll?
};

```

This is **hybrid inheritance**, and it is a different problem from the first one: nobody wrote `roll` twice. `Result` inherits `Student` **twice over** — once down the `Test` path and once down the `Sports` path — so a `Result` object physically contains **two** copies of `roll`. Even `Test::roll` would work but is wrong in spirit: a student has one roll number, not two.

Fix — the virtual base class. Mark the inheritance `virtual` at *both* of the middle classes:

```
class Student { protected: int roll; };

class Test    : virtual public Student { ... };
class Sports : public virtual Student { ... };    // order of the two
                                                    // keywords does
not matter
class Result : public Test, public Sports {
public:
    void show() { cout << roll; }    // fine -- exactly one roll now
};
```

One shared Student sub-object instead of two

What `virtual` actually does here. It tells the compiler: however many paths reach `Student`, make only **one** copy of it in the final object and let every path share it. The keyword goes on the classes that do the *intermediate* inheriting, not on `Result` — by the time you are writing `Result` it is already too late.

THE CONSEQUENCE PEOPLE GET ASKED ABOUT

With a virtual base, **the most derived class constructs it**. `Result`'s constructor calls `Student`'s directly; the calls that `Test` and `Sports` would normally make are **skipped**. That is the only way to guarantee the shared sub-object is built exactly once, and it surprises everybody the first time they hit it.

Two more one-liners worth keeping: **virtual base class** ≠ **virtual function** — same keyword, unrelated jobs. And an **abstract class** is one with a pure virtual function (`virtual void f() = 0;`), which cannot be instantiated and exists only to be inherited from.

WHY THERE IS NO PAST PAPER HERE

AND WHAT WE DID INSTEAD

The papers folder has four subjects in it, and this is not one of them. DSP, DComm, DCN and DIP all have at least one scanned mid-sem; C++ has none. We could have invented one and called it 2024. We did not.

So the page you have just read is built the only honest way it can be: from the eight lecture decks, from the shape the other four mid-sems take at this institute — 30 marks, ninety minutes, four questions, one of them “attempt any three” — and from the observation that a programming paper asks you to *read and write code* rather than to describe it.

What that means for you. Use the predicted paper as a revision set with an unusually good hit rate on topics, and an unknown one on exact wording. If a past paper turns up, this page gets rebuilt around it the same day — and the first thing that would change is the mark split, which is the one number here we are genuinely guessing at.

Everything else on this page is checkable. Every program has been traced statement by statement, every predicted output is derived in the solution rather than asserted, and where the behaviour is undefined by the language — the double `delete` in Q-3(b), the wrong `delete` in Q-2(d) — it says so instead of inventing a number.

[The notes →](#) · [The 40-question quiz →](#)

CS201 · OBJECT ORIENTED PROGRAMMING · PAPERS WORKED BY HAND AND CHECKED NUMERICALLY
THE PREDICTED PAPER IS OUR OWN READING OF THE PATTERN — IT IS NOT AN OFFICIAL ANYTHING, AND
IT IS NOT A LEAK
BACK TO TU CHAHIYE